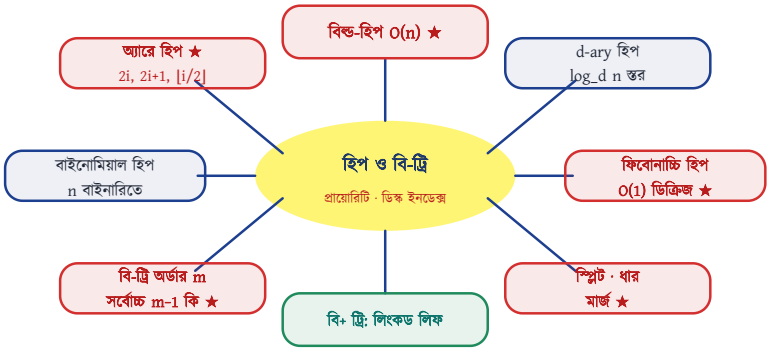
উন্নত ডেটা স্ট্রাকচার: হিপ, ফিবোনাচ্চি হিপ, বি-ট্রি

বাইনারি হিপ · প্রায়োরিটি কিউ · d-ary ও বাইনোমিয়াল হিপ · ফিবোনাচ্চি হিপ · বি-ট্রি · বি+ ট্রি: এগারো পাতায় পরীক্ষার প্রস্তুতি

এগারো পাতা: অ্যারেতে রাখা হিপ ও তার ইনডেক্স সূত্র, প্রতিটি হিপ অপারেশন ও তার খরচ, বিল্ড-হিপ কেন লিনিয়ার, ফিবোনাচ্চি হিপ পর্যন্ত হিপ পরিবার, তারপর ডিস্ক-বান্ধব বি-ট্রি, তার ইনসার্ট/ডিলিটের নিয়ম এবং প্রতিটি ডেটাবেস ইনডেক্সে ব্যবহৃত বি+ ট্রি।



↑ পুরো টপিক এক নজরে: লাল = বেশি আসে, সবুজ = ডেটাবেসের সংযোগ

সিলেবাসের যে অংশ

ডেটা স্ট্রাকচার ও অ্যালগরিদমের সিলেবাস লাইন: "Internal data representation; Abstract data types; Elementary data structures: arrays, lists, stacks, queues, trees, graphs; Advanced data Structures: heaps, Fibonacci heaps, B-trees;" এই সেটে **অ্যাডভান্সড** অংশটি পুরোপুরি নেওয়া হয়েছে, সঙ্গে বি+ ট্রি — যা প্রতিটি পরীক্ষায় এর সাথে জোড়া হিসেবে আসে। ADT ও প্রাথমিক স্ট্রাকচার এই অধ্যায়ের প্রথম লেসনে; **হিপসর্ট**, সার্চিং ও হ্যাশিং সর্টিং লেসনে, আর **ডাইকস্ট্রা / প্রিম** গ্রাফ-অ্যালগরিদম লেসনে — এখানে সেগুলো এসেছে কেবল যেখানে হিপই তাদের রানিং টাইমের কারণ।

হাই-ইল্ড তালিকা

① হিপ = সম্পূর্ণ ট্রি + হিপ ক্রম
② ১-ভিত্তিক: 2i, 2i+1, প্যারেন্ট ⌊i/2⌋
③ ০-ভিত্তিক: 2i+1, 2i+2, ⌊(i−1)/2⌋
④ ইনসার্ট / এক্সট্র্যাক্ট: O(log n)
⑤ **বিল্ড-হিপ: O(n) ★**
⑥ হিপে সার্চ: O(n)
⑦ ফিবো হিপ ডিক্রিজ-কি O(1) অ্যামর্টা.
⑧ ডাইকস্ট্রা + ফিবো: O(E + V log V)
⑨ অর্ডার m: ≤ m−1 কি, ≤ m চাইল্ড
⑩ নন-রুট ≥ ⌈m/2⌉ চাইল্ড ★
⑪ বি-ট্রি বাড়ে রুটে
⑫ বি+ ট্রি: ডেটা লিংকড লিফে

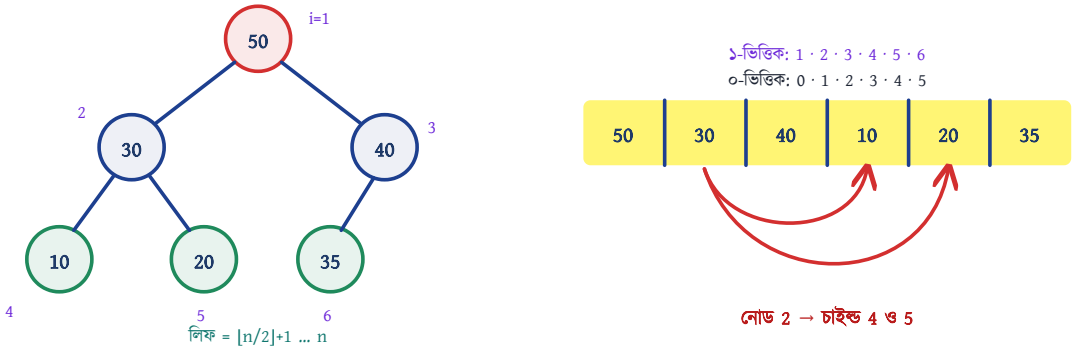
সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ বাইনারি হিপ, অ্যারে সূত্র, d-ary ... পৃ. ২
✓ হিপ অপারেশন, সিফট-আপ/ডাউন ... পৃ. ৩
✓ বিল্ড-হিপ O(n), হিপসর্ট, PQ ... পৃ. ৪
✓ বাইনোমিয়াল ও ফিবোনাচ্চি হিপ ... পৃ. ৫
✓ হিপের খরচের তুলনা, ডাইকস্ট্রা ... পৃ. ৬
✓ বি-ট্রি: অর্ডার m বনাম ডিগ্রি t ... পৃ. ৭
✓ বি-ট্রি ইনসার্ট ও ডিলিট ... পৃ. ৮
✓ বি+ ট্রি ও বড় তুলনা ... পৃ. ৯
✓ রাতের পরীক্ষার পাতা ... পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১১

১ বাইনারি হিপ: অ্যারেতে রাখা একটি সম্পূর্ণ ট্রি

সংজ্ঞা: **বাইনারি হিপ** (Binary Heap) এমন একটি বাইনারি ট্রি যা একসাথে **দুটি** শর্ত পূরণ করে। **আকার:** এটি একটি **সম্পূর্ণ বাইনারি ট্রি** (Complete Binary Tree) — শেষটি ছাড়া প্রতিটি স্তর পূর্ণ, আর শেষ স্তর ভরা হয় **বাম দিক থেকে**। **ক্রম (হিপ বৈশিষ্ট্য): ম্যাক্স-হিপে** (Max-Heap) প্রতিটি প্যারেন্ট ≥ তার চাইল্ড, তাই **সর্বোচ্চ মান রুটে**; **মিন-হিপে** (Min-Heap) প্রতিটি প্যারেন্ট ≤ তার চাইল্ড, তাই সর্বনিম্ন মান রুটে। বাম আর ডান চাইল্ডের মধ্যে কোনো ক্রমের কথা নেই — হিপ **বাইনারি সার্চ ট্রি নয়**।



একই ম্যাক্স-হিপ, ট্রি হিসেবে ও লেভেল-অর্ডার অ্যারে হিসেবে — কোনো পয়েন্টার রাখা হয় না

রাশি	রুট ইনডেক্স 1-এ (CLRS)	রুট ইনডেক্স 0-তে (C / Java অ্যারে)
i-এর বাম চাইল্ড	**2i**	2i + 1
i-এর ডান চাইল্ড	2i + 1	2i + 2
i-এর প্যারেন্ট	⌊i / 2⌋	⌊(i − 1) / 2⌋
লিফ	ইনডেক্স ⌊n/2⌋ + 1 … n	ইনডেক্স ⌊n/2⌋ … n − 1
শেষ ইন্টারনাল নোড	⌊n/2⌋	⌊n/2⌋ − 1

আকারের তথ্য ✍ সংখ্যাগুলো: n-নোডের হিপের উচ্চতা = $\lfloor \log_2 n \rfloor$। h উচ্চতার হিপে নোড থাকে 2^h থেকে 2^(h+1) − 1-টি। লিফ ⌊n/2⌋-টি, আর h উচ্চতার নোড সর্বোচ্চ ⌈n / 2^(h+1)⌉-টি। ট্রি সম্পূর্ণ বলে অ্যারেতে **কোনো ফাঁক নেই** — ইনডেক্সের হিসাব কাজ করার একমাত্র কারণ এটিই।

মনে রাখার কৌশল: **"নিচে নামতে দ্বিগুণ, উপরে উঠতে অর্ধেক।"** রুট ১-এ থাকলে বাম চাইল্ড = প্যারেন্টের দ্বিগুণ, ডান চাইল্ড তার এক বেশি; প্যারেন্ট = চাইল্ডের অর্ধেক (ফ্লোর)। ০-ভিত্তিক অ্যারেতে **"এক ঘর সরাও"**: দ্বিগুণ করার আগে ১ যোগ, অর্ধেক করার আগে ১ বিয়োগ।

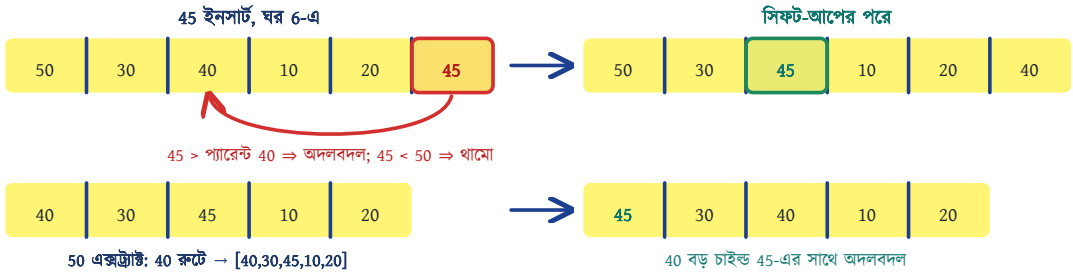
পরীক্ষার ফাঁদ ✍ তিনটি: **১.** আগে প্রশ্নের ইনডেক্সিং দেখুন: 2i, 2i+1 উত্তর **কেবল** রুট ১-এ থাকলে; রুট ০-তে হলে 2i+1, 2i+2। **২.** ম্যাক্স-হিপে **সর্বনিম্ন** মান কোনো একটি **লিফে** থাকে (অবস্থান ⌊n/2⌋+1 … n), অবশ্যই শেষ ঘরে নয়। **৩. অবরোহী** (descending) ক্রমে সাজানো অ্যারে সবসময় বৈধ ম্যাক্স-হিপ, কিন্তু বৈধ ম্যাক্স-হিপ সাধারণত সাজানো **নয়**।

সাধারণীকরণ: d-ary হিপ: একই সম্পূর্ণ-ট্রি-অ্যারের ধারণা, কিন্তু প্রতিটি নোডের d-টি চাইল্ড। ০-ভিত্তিক: i-এর চাইল্ড d·i + 1 … d·i + d, প্যারেন্ট ⌊(i − 1)/d⌋। উচ্চতা ≈ $\log_d n$, তাই **ইনসার্ট / ডিক্রিজ-কি** = $O(\log_d n)$ (ওঠার স্তর কম) কিন্তু **এক্সট্র্যাক্ট-মিন** = $O(d \cdot \log_d n)$ (প্রতি স্তরে d-টি চাইল্ডের তুলনা)। ডিক্রিজ-কি বেশি আর এক্সট্র্যাক্ট কম হলে বড় d উপযোগী — যেমন ঘন গ্রাফে ডাইকস্ট্রা।

২ হিপ অপারেশন: সিফট-আপ, সিফট-ডাউন ও প্রতিটির খরচ

ইনসার্ট = সিফট-আপ (Sift-up): নতুন কি-কে **পরবর্তী খালি ঘরে** (ইনডেক্স n+1) রাখুন যাতে ট্রি সম্পূর্ণ থাকে, তারপর **যতক্ষণ প্যারেন্টের চেয়ে বড়, ততক্ষণ প্যারেন্টের সাথে অদলবদল** (ম্যাক্স-হিপ)। প্রতি স্তরে সর্বোচ্চ একটি অদলবদল ⇒ **O(log n)**। ম্যাক্স-হিপে **ইনক্রিজ-কি** (মিন-হিপে ডিক্রিজ-কি) একই সিফট-আপ।

এক্সট্র্যাক্ট-ম্যাক্স = সিফট-ডাউন (Heapify): রুট সরিয়ে রাখুন, **শেষ উপাদানটি রুটে আনুন**, n এক কমান, তারপর **বড় চাইল্ডের সাথে অদলবদল** করুন যতক্ষণ না দুই চাইল্ডই ছোট হয়। প্রতি স্তরে দুটি তুলনা ⇒ **O(log n)**। এই সিফট-ডাউনের নাম **MAX-HEAPIFY**।



একই অ্যারেতে একটি ইনসার্ট (উপরে) ও একটি এক্সট্র্যাক্ট-ম্যাক্স (নিচে), হাতে-কলমে

অপারেশন (ম্যাক্স-হিপ, n-টি কি)	সময়	কীভাবে
ফাইন্ড-ম্যাক্স / পিক	**O(1)**	A[1] পড়া
ইনসার্ট	O(log n)	শেষে যোগ, সিফট-আপ
এক্সট্র্যাক্ট-ম্যাক্স	O(log n)	শেষটি → রুট, সিফট-ডাউন
ইনক্রিজ-কি (মিন-হিপে ডিক্রিজ-কি)	O(log n)	মান বদল, সিফট-আপ
জানা ইনডেক্সে ডিলিট	O(log n)	শেষটির সাথে অদলবদল, সিফট আপ *বা* ডাউন
একটি নোডে MAX-HEAPIFY	**O(log n)**	একটি সিফট-ডাউন (h উচ্চতার নোডে O(h))
n-টি কি-র BUILD-HEAP	**O(n)**	⌊n/2⌋ ... 1 হিপিফাই (পরের পাতা)
যেকোনো কি সার্চ	O(n)	সিবলিংদের মধ্যে কোনো ক্রম নেই
দুটি হিপ মার্জ	O(n)	অ্যারে জোড়া দিয়ে বিল্ড-হিপ

```
MAX-HEAPIFY(A, i, n):   // 1-based
  l = 2i ; r = 2i + 1 ; big = i
  if l ≤ n and A[l] > A[big]: big = l
  if r ≤ n and A[r] > A[big]: big = r
  if big ≠ i:
      swap A[i], A[big]
      MAX-HEAPIFY(A, big, n)
```

পরীক্ষার ফাঁদ ✍ হিপিফাই বনাম বিল্ড-হিপ: "হিপিফাই O(n) সময় নেয়" — এক পরিচিত গোলমাল। **HEAPIFY** (একটি নোড থেকে একটি সিফট-ডাউন) **O(log n)**। **BUILD-HEAP** (নিচ থেকে উপরে প্রতিটি ইন্টারনাল নোড হিপিফাই) **O(n)**। প্রশ্ন কোনটি বোঝাচ্ছে পড়ে নিন। আরও: হিপ ম্যাক্স দেয় O(1)-এ, কিন্তু অন্য যেকোনো কি **সার্চ** করতে O(n)।

৩ O(n)-এ বিল্ড-হিপ, হিপসর্ট ও প্রায়োরিটি কিউ

BUILD-MAX-HEAP: n-টি কি যেমন আছে তেমনি অ্যারেতে রাখুন, তারপর **i = ⌊n/2⌋ থেকে 1 পর্যন্ত** `MAX-HEAPIFY(A, i)` ডাকুন। লিফগুলো (⌊n/2⌋+1 ... n) আগে থেকেই এক-নোডের হিপ, তাই বাদ; নিচ থেকে উপরে যাওয়ায় i-কে হিপিফাই করার সময় তার দুই সাবট্রিই হিপ থাকে।

কেন O(n), O(n log n) নয় ✍️ ৪ নম্বর: h উচ্চতার নোডে হিপিফাইয়ের খরচ **O(h)**, আর এমন নোড সর্বোচ্চ `⌈n / 2^(h+1)⌉`-টি। তাই
`T(n) = Σ_(h=0..⌊log n⌋) ⌈n/2^(h+1)⌉ · O(h) = O(n · Σ h/2^h)`।
যেহেতু `Σ_(h≥0) h/2^h = 2`, **T(n) = O(2n) = O(n)**।
অন্তর্দৃষ্টি: **অর্ধেক নোডই লিফ**, তাদের কোনো কাজ নেই; এক-চতুর্থাংশ সর্বোচ্চ এক স্তর নামে, এক-অষ্টমাংশ সর্বোচ্চ দুই — দামি নোড বিরল।

সমাধানকৃত উদাহরণ ✍️: `[3, 9, 2, 1, 4, 5]` থেকে ম্যাক্স-হিপ বানান, n = ৬, তাই শুরু ⌊6/2⌋ = 3 থেকে।

i = 3: 2 বনাম চাইল্ড 5 → অদলবদল → `[3, 9, 5, 1, 4, 2]`

i = 2: 9 বনাম 1, 4 → অদলবদল নেই

i = 1: 3 বনাম 9, 5 → 9-এর সাথে অদলবদল → `[9, 3, 5, 1, 4, 2]`; 3 এখন ঘর 2-এ, বনাম 1, 4 → 4-এর সাথে অদলবদল → **`[9, 4, 5, 1, 3, 2]`**

যাচাই: 9 ≥ 4, 5 · 4 ≥ 1, 3 · 5 ≥ 2 ✓ — **৩টি অদলবদল**।

পরীক্ষার ফাঁদ ✍️ বানানোর দুই পথ: n-টি কি **একটি একটি করে** ইনসার্ট করলে (উপর থেকে, প্রতিবার সিফট-আপ) সবচেয়ে খারাপ ক্ষেত্রে **O(n log n)** — যেমন ম্যাক্স-হিপে ঊর্ধ্বক্রমে কি আসলে প্রত্যেকটি রুট পর্যন্ত ওঠে। নিচ-থেকে-উপরের **BUILD-HEAP হলো O(n)**। দুটিই বৈধ হিপ দেয়, কিন্তু **অবশ্যই একই অ্যারে নয়**; "বিল্ড করার পরের হিপ" চাইলে নিচ-থেকে-উপরের পদ্ধতি বোঝায়, যদি না প্রশ্নে "একটি একটি করে ইনসার্ট" বলা থাকে।

হিপসর্ট — সর্টিংয়ের সাথে যোগসূত্র: ম্যাক্স-হিপ বানান, তারপর n - 1 বার: `A[1] ↔ A[k]` অদলবদল, হিপ এক ঘর ছোট, `MAX-HEAPIFY(A, 1)`। অ্যারের বাঁ দিকে হিপ, ডান দিকে ক্রমশ বাড়তে থাকা সাজানো অংশ। বিল্ড-হিপ O(n) + (n - 1)-টি এক্সট্র্যাক্ট-ম্যাক্স, প্রতিটি O(log n) ⇒ **বেস্ট, অ্যাভারেজ ও ওয়ার্স্ট — সব ক্ষেত্রে O(n log n)**। **ইন-প্লেস** (O(1) বাড়তি জায়গা) কিন্তু **স্টেবল নয়**। ম্যাক্স-হিপ ঊর্ধ্বক্রমে সাজায়। কুইকসর্ট ও মার্জ সর্টের সাথে পূর্ণ তুলনা এই অধ্যায়ের সর্টিং লেসনে।

প্রায়োরিটি কিউ (ADT): এমন সংগ্রহ যেখানে প্রতিটি উপাদানের একটি **অগ্রাধিকার (priority)** আছে এবং অপারেশন `insert`, `find-max`, `extract-max`, `increase-key` (ম্যাক্স-PQ) বা এদের মিন সংস্করণ। হিপই এর **আদর্শ বাস্তবায়ন**: অসাজানো অ্যারেতে ইনসার্ট O(1) কিন্তু এক্সট্র্যাক্ট O(n); সাজানো অ্যারেতে উল্টো; হিপে দুটিই O(log n)।

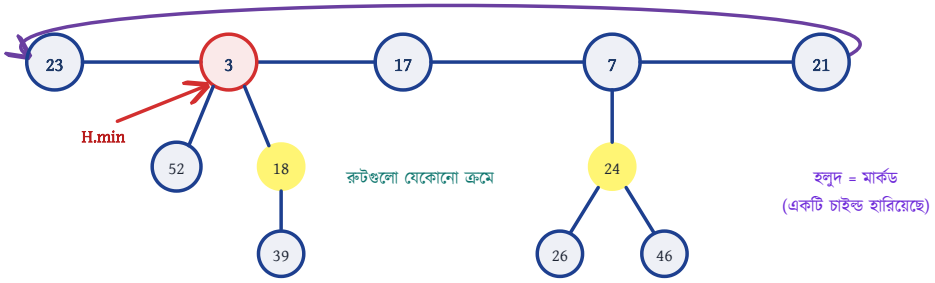
হিপ / প্রায়োরিটি কিউ কোথায় ব্যবহৃত	যে হিপ অপারেশন গুরুত্বপূর্ণ
CPU / জব শিডিউলিং, ইভেন্ট-চালিত সিমুলেশন	পরবর্তী ডেডলাইন বা ইভেন্ট সময়ের এক্সট্র্যাক্ট-মিন
ডাইকস্ট্রা, প্রিম (গ্রাফ-অ্যালগরিদম লেসন)	প্রতি ভার্টেক্সে একবার এক্সট্র্যাক্ট-মিন, প্রতি এজে একবার **ডিক্রিজ-কি**
হাফম্যান কোডিং, k-ওয়ে মার্জ, n-এর মধ্যে টপ-k	বারবার এক্সট্র্যাক্ট-মিন; k আকারের হিপে টপ-k **O(n log k)**

৪ মার্জযোগ্য হিপ: বাইনোমিয়াল হিপ ও ফিবোনাচ্চি হিপ

বাইনোমিয়াল হিপ (Binomial Heap): B_0 একটি নোড; B_k = **দুটি B_{k-1}, একটির রুট অন্যটির নিচে ঝোলানো**। B_k-তে 2^k**-টি নোড**, **উচ্চতা** k, গভীরতা i-তে $C(k, i)$-টি নোড, এবং রুটের **ডিগ্রি** k। বাইনোমিয়াল হিপ হলো হিপ-ক্রমের বাইনোমিয়াল ট্রির তালিকা, **প্রতিটি** k**-এর জন্য সর্বোচ্চ একটি** B_k — ট্রিগুলো n-এর ১-বিটের সাথে মেলে: $13 = 1101_2 \Rightarrow B_3, B_2, B_0$, তাই সর্বোচ্চ $\lfloor \log_2 n \rfloor + 1$**-টি ট্রি**।

মনে রাখার কৌশল: **"বাইনোমিয়াল হিপ মানে n-কে বাইনারিতে লেখা; ইউনিয়ন মানে বাইনারি যোগ।"** দুটি B_k যেন একই কলামে দুটি ১-বিট: জুড়ে একটি B_{k+1} বানাও আর **হাতে রাখো (carry)** — ইউনিয়ন $O(\log n)$। ইনসার্ট মানে "১ যোগ": লম্বা ক্যারি-চেইনে সবচেয়ে খারাপ ক্ষেত্রে $O(\log n)$, কিন্তু বাইনারি কাউন্টারের মতো **অ্যামর্টাইজড** $O(1)$।

ফিবোনাচ্চি হিপ (Fibonacci Heap): (ফ্রেডম্যান ও টারজান, ১৯৮৪) **হিপ-ক্রমের ট্রির** এমন সংগ্রহ যাদের রুটগুলো একটি **বৃত্তাকার, দ্বিমুখী লিংকড রুট লিস্টে** থাকে, আর একটি পয়েন্টার **H.min** সবচেয়ে ছোট রুটকে নির্দেশ করে। ট্রিগুলো বাইনোমিয়াল হতে হবে **না**, একাধিক ট্রির একই ডিগ্রি থাকতে পারে — কাঠামোটি **অলস (lazy)**: এক্সট্র্যাক্ট-মিন বাধ্য না করা পর্যন্ত গোছানো স্থগিত রাখে। প্রতিটি নোডে থাকে **কি, ডিগ্রি, প্যারেন্ট, একটি চাইল্ড পয়েন্টার, বাম/ডান সিবলিং পয়েন্টার** (চাইল্ডরাও নিজস্ব বৃত্তাকার লিস্ট, তাই যেকোনো জোড়া বা লিস্ট সংযোজন $O(1)$) এবং একটি বুলিয়ান **মার্ক** = "শেষবার অন্য নোডের চাইল্ড হওয়ার পর থেকে একটি চাইল্ড হারিয়েছে"।



রুট লিস্টে পাঁচটি ট্রি; 23, 17 ও 21 — তিনটিরই ডিগ্রি 0 — একই ডিগ্রি থাকা চলে

সস্তা অপারেশনগুলো: **ইনসার্ট:** এক-নোডের ট্রি বানিয়ে রুট লিস্টে জুড়ে দিন, H.min হালনাগাদ — $O(1)$, কোনো গোছানো নেই। **ইউনিয়ন:** দুই রুট লিস্ট জোড়া দিন, ছোট মিন রাখুন — $O(1)$। **ফাইন্ড-মিন:** H.min পড়ুন — $O(1)$।

এক্সট্র্যাক্ট-মিন = পরিষ্কারের কাজ: H.min সরান, তার চাইল্ডদের রুট লিস্টে আনুন, তারপর **কনসলিডেট (consolidate): সমান ডিগ্রির দুটি রুট জুড়ুন** (বড় কি ছোটটির নিচে) যতক্ষণ না সব ডিগ্রি আলাদা হয়, ডিগ্রি-ইনডেক্সড অ্যারে দিয়ে; তারপর রুটগুলো দেখে নতুন মিন খুঁজুন।

ডিক্রিজ-কি = কাট + ক্যাসকেডিং কাট: x-এর কি কমান। এখন যদি তা প্যারেন্ট y-এর চেয়ে ছোট হয়, x-কে **কাট** করুন: সাবট্রিসহ রুট লিস্টে সরিয়ে আনমার্ক করুন। তারপর: y **আনমার্কড** হলে মার্ক করে থামুন; y **আগে থেকেই মার্কড** হলে y-কেও কাট করুন এবং y-এর প্যারেন্টে একই কাজ করুন — এটাই **ক্যাসকেডিং কাট** — আনমার্কড নোড বা রুটে পৌঁছানো পর্যন্ত। এক লাইনে: **নন-রুট নোড একটি চাইল্ড হারাতে পারে; দ্বিতীয়টি হারালে নিজেই কাট হয়।**

পরীক্ষার ফাঁদ ✍: এই হিপগুলো কেন আছে: বাইনারি হিপ $O(n)$-এর চেয়ে দ্রুত **মার্জ** করতে পারে না। বাইনোমিয়াল ও ফিবোনাচ্চি হিপ পয়েন্টার-ভিত্তিক **মার্জযোগ্য হিপ (mergeable heap)** (লেফটিস্ট ও স্কিউ হিপও তাই)।

৫ হিপের খরচের তুলনা, আর ডাইকস্ট্রা কেন ফিবোনাচ্চি হিপ চায়

সমাধানকৃত উদাহরণ ✍️ একটি ক্যাসকেডিং কাট: আগের পাতার ট্রিতে 24 ও 18 মার্কড। **46 → 15 ডিক্রিজ:** 15 < প্যারেন্ট 24 ⇒ 15-কে রুট লিস্টে কাট; 24 মার্কড ⇒ 24-কেও (26 সহ) কাট; তার প্যারেন্ট 7 একটি রুট ⇒ থামো। **39 → 1 ডিক্রিজ:** 1 < 18 ⇒ 1-কে কাট; 18 মার্কড ⇒ 18-কে কাট; তার প্যারেন্ট 3 রুট ⇒ থামো; **H.min = 1**। প্রতিটি কাট O(1); ক্যাসকেডের খরচ মেটে সে যেসব মার্ক মুছে দেয় তা দিয়ে।

পোটেনশিয়াল ফাংশন: $\Phi(H) = t(H) + 2 \cdot m(H)$: t = রুট লিস্টে ট্রির সংখ্যা, m = মার্কড নোডের সংখ্যা। কনসলিডেশনের খরচ মেটে সে যত ট্রি কমায় তা দিয়ে; প্রতিটি ক্যাসকেডিং কাট একটি নোড **আনমার্ক** করে (−2) ও একটি ট্রি যোগ করে (+1), তাই নিজের খরচ নিজেই মেটায় ⇒ ডিক্রিজ-কি **অ্যামর্টাইজড O(1)**।

"ফিবোনাচ্চি" নাম কোথা থেকে: কাট হওয়ার আগে একটি নোড সর্বোচ্চ একটি চাইল্ড হারায়, তাই ডিগ্রি k-এর নোডের সাবট্রিতে অন্তত $F_{k+2} \geq \varphi^k$-টি নোড ($\varphi \approx 1.618$)। ফলে সর্বোচ্চ ডিগ্রি $D(n) \leq \lfloor \log_\varphi n \rfloor \approx 1.44 \log_2 n$ — যা এক্সট্র্যাক্ট-মিনকে অ্যামর্টাইজড O(log n) করে।

অপারেশন	বাইনারি হিপ (ওয়ার্স্ট)	বাইনোমিয়াল হিপ (ওয়ার্স্ট)	ফিবোনাচ্চি হিপ (অ্যামর্টাইজড)
ইনসার্ট	O(log n)	O(log n); অ্যামর্টাইজড O(1)	**O(1)**
ফাইন্ড-মিন	O(1)	O(log n); মিন পয়েন্টারে O(1)	O(1)
এক্সট্র্যাক্ট-মিন	O(log n)	O(log n)	O(log n)
ইউনিয়ন / মার্জ	O(n)	**O(log n)**	**O(1)**
ডিক্রিজ-কি	O(log n)	O(log n)	**O(1)**
ডিলিট (−∞ পর্যন্ত কমিয়ে এক্সট্র্যাক্ট)	O(log n)	O(log n)	O(log n)

ডাইকস্ট্রা / প্রিম: হিপ কল গুনুন ✍️: V-টি ভার্টেক্স ও E-টি এজের গ্রাফে দুটি অ্যালগরিদমই করে **V-টি ইনসার্ট**, **V-টি এক্সট্র্যাক্ট-মিন** এবং সর্বোচ্চ **E-টি ডিক্রিজ-কি**। **অ্যারে (হিপ ছাড়া):** $V \times O(V) \Rightarrow O(V^2)$ — ঘন গ্রাফে সেরা। **বাইনারি হিপ:** $(V + E) \times O(\log V) \Rightarrow O((V + E) \log V)$। **ফিবোনাচ্চি হিপ:** $V \times O(\log V) + E \times O(1) \Rightarrow O(E + V \log V)$। লাভটা **E-টি ডিক্রিজ-কি**-তে: $O(\log V) \to O(1)$। (অ্যালগরিদম: গ্রাফ-অ্যালগরিদম লেসন।)

পরীক্ষার ফাঁদ ✍️ দুটি: **১.** ফিবোনাচ্চি হিপের সীমাগুলো **অ্যামর্টাইজড**: একটি এক্সট্র্যাক্ট-মিনের প্রকৃত খরচ **O(n)** হতে পারে (n-টি ইনসার্টের পর n-টি রুট), একটি ডিক্রিজ-কি O(log n)। **২.** এখানে কোনো হিপেই এক্সট্র্যাক্ট-মিন O(1) **নয়**।

মনে রাখার কৌশল: **"ফিবোনাচ্চি অলস: মিনিমাম বের করা ছাড়া সবই O(1)।"** ইনসার্ট, ইউনিয়ন, ফাইন্ড-মিন, ডিক্রিজ-কি — O(1); এক্সট্র্যাক্ট-মিন ও ডিলিট — O(log n), কারণ তখনই অলস হিপ রুট লিস্ট গোছায়। আর **"এক চাইল্ড হারালে মার্ক, দুই হারালে কাট।"** বাস্তবে বাইনারি হিপ প্রায়ই দ্রুততর।

৬ বি-ট্রি: অর্ডার m, মিনিমাম ডিগ্রি t ও উচ্চতার সীমা

সংজ্ঞা: **বি-ট্রি (B-Tree)** (বায়ার ও ম্যাকক্রেইট, ১৯৭২) ডিস্কের জন্য নকশা করা একটি **সুষম বহুমুখী সার্চ ট্রি (balanced multiway search tree)**: এক নোড = এক ডিস্ক ব্লক, যাতে অনেকগুলো সাজানো কি থাকে। **k-টি কি-যুক্ত নোডের k + 1-টি চাইল্ড**, আর কিগুলো চাইল্ডদের কি-পরিসর আলাদা করে। **সব লিফ একই স্তরে**, তাই প্রতিটি সার্চ সমান সংখ্যক ব্লক পড়ে। ফ্যান-আউট বড় বলে উচ্চতা খুব কম, এবং সার্চ, ইনসার্ট ও ডিলিট সবই O(log n)।

অর্ডার m (নুথ) — পরীক্ষার রীতি: ১. প্রতিটি নোডে **সর্বোচ্চ m-টি চাইল্ড** ⇒ সর্বোচ্চ m − 1-টি কি।
২. রুট ছাড়া প্রতিটি ইন্টারনাল নোডে **অন্তত ⌈m/2⌉-টি চাইল্ড** ⇒ অন্তত ⌈m/2⌉ − 1-টি কি (নন-রুট লিফেও একই ন্যূনতম কি)।
৩. রুট লিফ না হলে তার **অন্তত ২টি চাইল্ড** ⇒ অন্তত ১টি কি।
৪. সব লিফ একই স্তরে।

মিনিমাম ডিগ্রি t (CLRS): t ≥ 2। প্রতিটি নন-রুট নোডে **t − 1 থেকে 2t − 1-টি কি**, অর্থাৎ ইন্টারনাল হলে **t থেকে 2t-টি চাইল্ড**। রুটে অন্তত ১টি কি। 2t − 1-টি কি-যুক্ত নোড **পূর্ণ (full)**।
অনুবাদ: অর্ডার m = 2t, তাই CLRS-এর ট্রির অর্ডার সবসময় জোড়। t = 2 ⇒ অর্ডার 4 ⇒ **2-3-4 ট্রি**।

অর্ডার m	সর্বোচ্চ চাইল্ড	সর্বোচ্চ কি	ন্যূনতম চাইল্ড (নন-রুট)	ন্যূনতম কি (নন-রুট)	অন্য নাম
3	3	2	2	1	2-3 ট্রি
4	4	3	2	1	2-3-4 ট্রি (CLRS t = 2)
5	**5**	**4**	**3**	**2**	পরীক্ষার পরিচিত উদাহরণ (পরের পাতা)
m	m	m − 1	⌈m/2⌉	⌈m/2⌉ − 1	সাধারণ নিয়ম

উচ্চতার সীমা ✍️ প্রতিপাদন: h উচ্চতায় সবচেয়ে কম কি (CLRS): রুটে ১টি কি, তারপর 2, 2t, $2t^2$, … , $2t^{h-1}$-টি নোড, প্রতিটিতে t − 1-টি কি:
`n ≥ 1 + (t−1)·Σ 2t^(i−1) = 2t^h − 1`
⇒ **h ≤ log_t((n + 1)/2)**। অর্ডার-m রূপে: `h ≤ log_⌈m/2⌉((n+1)/2)`। h উচ্চতায় সর্বাধিক কি: সব নোড পূর্ণ ⇒ `m^(h+1) − 1`।

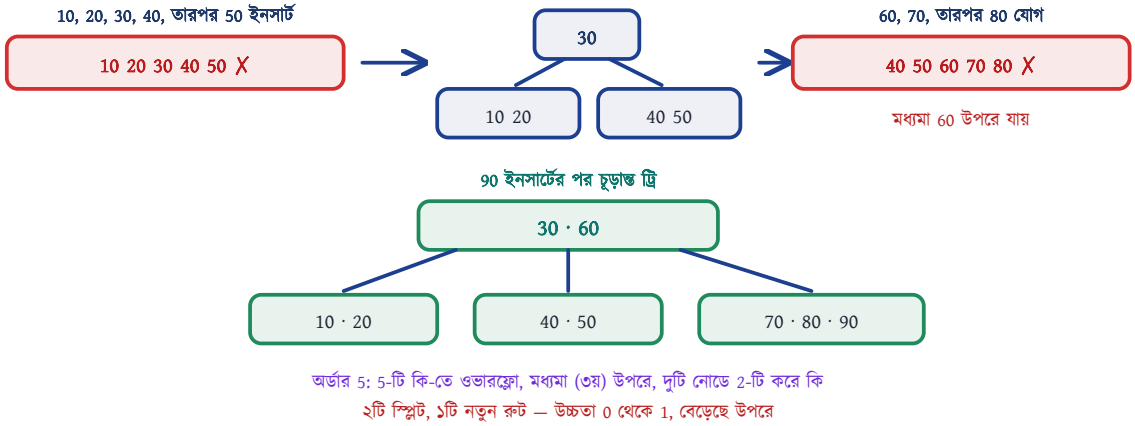
একটি সার্চের খরচ: রুট থেকে নামুন, লক্ষ্য কি যে দুই কি-র মাঝে পড়ে সেই চাইল্ডে যান। **ডিস্ক রিড = O(h) = O(log_t n)**। প্রতি নোডে CPU কাজ লিনিয়ার স্ক্যানে O(t) (বাইনারি সার্চে O(log t)) ⇒ মোট **O(t log_t n)** CPU। ডিস্ক রিডই প্রধান, তাই t এমনভাবে বাছা হয় যাতে এক নোড এক ব্লক ভরে।

পরীক্ষার ফাঁদ ✍️ কোন "অর্ডার"?: MCQ-তে "অর্ডার" প্রায় সবসময় **নুথের সর্বোচ্চ চাইল্ড সংখ্যা**: অর্ডার 5 ⇒ সর্বোচ্চ **4**-টি কি, নন-রুট নোডে ন্যূনতম **2**-টি কি। "5-টি কি" উত্তর দেবেন না। প্রশ্নে **মিনিমাম ডিগ্রি t** দিলে t − 1 … 2t − 1-টি কি ধরুন। আর ন্যূনতম হলো ⌈m/2⌉-টি চাইল্ড — অর্থাৎ ⌈m/2⌉ − 1-টি **কি**।

৭ বি-ট্রিতে ইনসার্ট (স্প্লিট) ও ডিলিট (ধার / মার্জ)

ইনসার্ট: সঠিক **লিফ** পর্যন্ত নেমে কি-টি সাজানো অবস্থানে বসান। লিফে এখন **m-টি কি** হলে (ওভারফ্লো) **স্প্লিট (split) করুন: মধ্যমা কি উপরে** প্যারেন্টে যায়, তার বাঁয়ের কিগুলো একটি নোড, ডানেরগুলো আরেকটি। প্যারেন্ট ওভারফ্লো হলে সেটিও স্প্লিট। রুট স্প্লিট হলে **নতুন রুট** তৈরি হয় — **ট্রি কেবল রুটে বাড়ে**, তাই সব লিফ এক স্তরে থাকে।

ডিলিট: **১.** ন্যূনতমের বেশি কি-যুক্ত লিফে কি থাকলে ⇒ শুধু সরিয়ে দিন। **২.** ইন্টারনাল নোডে কি থাকলে ⇒ তার **ইন-অর্ডার প্রিডিসেসর** (বাম সাবট্রির সবচেয়ে বড় কি) বা সাকসেসর দিয়ে প্রতিস্থাপন করে সেই কি লিফ থেকে মুছুন। **৩.** লিফ **আন্ডারফ্লো** হলে ($\lceil m/2 \rceil - 1$-এর কম কি) ⇒ বাড়তি কি-যুক্ত পাশের সিবলিং থেকে **ধার (borrow)** করুন (প্যারেন্টের মধ্য দিয়ে ঘুরিয়ে), নইলে সিবলিং ও প্যারেন্টের বিভাজক কি নিয়ে **মার্জ (merge)**। মার্জে প্যারেন্ট আন্ডারফ্লো হতে পারে; রুট খালি হলে ট্রি **এক স্তর কমে**।



অর্ডার-5 বি-ট্রিতে 10, 20, ..., 90 ইনসার্ট, ধাপে ধাপে

সেই ট্রিতে ডিলিট ✍️ অর্ডার 5, ন্যূনতম 2-টি কি: 40 ডিলিট (ধার): লিফ [40, 50] → [50] আন্ডারফ্লো। বাম সিবলিং [10, 20]-তে কেবল ন্যূনতম, ডান সিবলিং [70, 80, 90]-তে বাড়তি আছে ⇒ **ঘোরান:** বিভাজক 60 নিচে নামে, 70 উপরে ওঠে। ফল: রুট [30 | 70], লিফ [10, 20] [50, 60] [80, 90]।

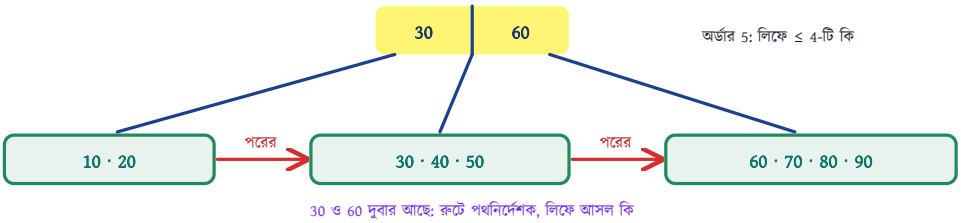
মূল ট্রি থেকে 20 ডিলিট (মার্জ): [10, 20] → [10] আন্ডারফ্লো; একমাত্র সিবলিং [40, 50]-এ বাড়তি নেই ⇒ **মার্জ** [10] + বিভাজক 30 + [40, 50] = [10, 30, 40, 50]। রুট হয় [60], লিফ [10, 30, 40, 50] ও [70, 80, 90] — তবুও বৈধ (রুটে ≥ ১টি কি)।

মনে রাখার কৌশল: **"বেশি ভরা? স্প্লিট করে মাঝেরটা উপরে ঠেলো। বেশি খালি? ধনী সিবলিং থেকে ধার, নইলে মার্জ করে বিভাজক নিচে টানো।"** ইনসার্ট কি **উপরে** ঠেলে; ডিলিট কি **নিচে** টানে। উচ্চতা বদলায় কেবল রুটে, দুই দিকেই।

পরীক্ষার ফাঁদ ✍️: **১.** বি-ট্রি কখনো লিফে বাড়ে না — নতুন স্তর আসে পুরোনো রুটের **উপরে**। **২. জোড়** অর্ডারে ওভারফ্লো নোডে জোড় সংখ্যক কি, তাই দুটি মাঝের কি; বইভেদে বাম- বা ডান-ঝোঁকা — আপনার পছন্দ উল্লেখ করুন। **৩.** CLRS ইনসার্ট করে **এক পাসে**, নামার পথে প্রতিটি পূর্ণ নোড স্প্লিট করে, তাই আর পিছু ফিরতে হয় না।

৮ বি+ ট্রি, ডেটাবেস ইনডেক্স ও বড় তুলনা

সংজ্ঞা: **বি+ ট্রি** (B+ Tree) বি-ট্রির একটি রূপ যেখানে **সব রেকর্ড (বা রেকর্ড পয়েন্টার) কেবল লিফে থাকে**; ইন্টারনাল নোডে থাকে শুধু পথনির্দেশক হিসেবে **কি-র কপি**, আর লিফগুলো **কি-ক্রমে লিংক করা** (সিকোয়েন্স সেট)।



একই নয়টি কি, একটি সঠিক দুই-স্তরের বি+ ট্রি: এক রুট, তিনটি লিংকড লিফ

বিষয়	বি-ট্রি	বি+ ট্রি
ডেটা / রেকর্ড পয়েন্টার কোথায়	**প্রতিটি** নোডে	**কেবল লিফে**
কি সংরক্ষণ	প্রতিটি কি ঠিক একবার	বিভাজক কি ইন্টারনাল নোডে পুনরাবৃত্ত
সার্চ	ইন্টারনাল নোডেই থামতে পারে	সবসময় লিফে শেষ; বেশি ফ্যান-আউট ⇒ খাটো ট্রি
রেঞ্জ কোয়েরি / পূর্ণ স্ক্যান	স্তর পেরিয়ে ইন-অর্ডার ট্রাভার্সাল	লিফ চেইন ধরে $O(\log n + k)$
লিফ স্প্লিট	মধ্যমা উপরে **সরে যায়**	মধ্যমা উপরে **কপি** হয় (লিফেও থাকে); ইন্টারনাল স্প্লিটে সরে

ফ্যান-আউটের হিসাব ✍️: 4 KB পেজ, 8-বাইট কি + 8-বাইট চাইল্ড পয়েন্টার ⇒ প্রতি নোডে প্রায় $4096 \div 16 = 256$-টি চাইল্ড। তিন স্তরে $256^3 = 16,777,216$-টি লিফ। রুট RAM-এ থাকলে ⇒ প্রায় ১.৬৮ কোটি কি-র যেকোনোটি **২-৩টি ডিস্ক রিডে**।

কোথায় ব্যবহৃত: **ডেটাবেস:** MySQL InnoDB, PostgreSQL, Oracle ও SQL Server সাধারণ ইনডেক্সে বি+ ট্রি ব্যবহার করে। **ফাইল সিস্টেম:** NTFS ডিরেক্টরি ইনডেক্স (বি+ ট্রি), ext4 ডিরেক্টরি (HTree, হ্যাশড বি-ট্রি), XFS ও Btrfs (বি-ট্রি)। বি+ ট্রি বনাম হ্যাশ ইনডেক্স: ডেটাবেস অধ্যায় দেখুন।

স্ট্রাকচার	ফাইন্ড-মিন / ম্যাক্স	যেকোনো কি সার্চ	ইনসার্ট	যার জন্য সেরা
বাইনারি হিপ	$O(1)$	$O(n)$	$O(\log n)$	প্রায়োরিটি কিউ, হিপসর্ট
ফিবোনাচ্চি হিপ	$O(1)$	$O(n)$	অ্যামর্টাইজড $O(1)$	অনেক ডিক্রিজ-কি (ডাইকস্ট্রা, প্রিম)
বি-ট্রি	$O(\log n)$	$O(\log n)$	$O(\log n)$	ডিস্কভিত্তিক সাজানো সংরক্ষণ
বি+ ট্রি	$O(\log n)$	$O(\log n)$	$O(\log n)$	DB ইনডেক্স, রেঞ্জ কোয়েরি

ফাঁদ: হিপ সার্চ স্ট্রাকচার নয় — কেবল ফাইন্ড-**মিন** $O(1)$; "ফিবোনাচ্চি হিপে $O(1)$ সার্চ" ভুল।

পরীক্ষার আগের রাতের পাতা

তথ্য-পাতা:

হিপ = সম্পূর্ণ ট্রি + হিপ ক্রম

ম্যাক্স-হিপ: ম্যাক্স রুটে, মিন লিফে

১-ভিত্তিক: $2i$, $2i+1$, $\lfloor i/2 \rfloor$

০-ভিত্তিক: $2i+1$, $2i+2$, $\lfloor (i-1)/2 \rfloor$

উচ্চতা = $\lfloor \log_2 n \rfloor$

লিফ: $\lfloor n/2 \rfloor + 1 \ldots n$ (মোট $\lceil n/2 \rceil$)

ইনসার্ট / এক্সট্র্যাক্ট / হিপিফাই: $O(\log n)$

বিল্ড-হিপ: $O(n)$

হিপে সার্চ: $O(n)$

হিপসর্ট: $O(n \log n)$, ইন-প্লেস, আনস্টেবল

d-ary: ইনসার্ট $O(\log_d n)$

d-ary: এক্সট্র্যাক্ট $O(d \log_d n)$

B_k: 2^k নোড, উচ্চতা k, $C(k,i)$

বাইনোমিয়াল হিপ ↔ n বাইনারিতে

বাইনোমিয়াল ইউনিয়ন: $O(\log n)$

বাইনারি-হিপ ইউনিয়ন: $O(n)$

ফিবো: ইনসার্ট/ইউনিয়ন/ফাইন্ড/ডিক্রিজ $O(1)$

ফিবো: এক্সট্র্যাক্ট-মিন/ডিলিট $O(\log n)$

Φ = ট্রি + 2 × মার্কড

$D(n) \le \log_\varphi n$

ডাইকস্ট্রা + ফিবো: $O(E + V \log V)$

ডাইকস্ট্রা + বাইনারি: $O((V+E) \log V)$

অর্ডার m: ≤ m চাইল্ড, ≤ $m-1$ কি

নন-রুট ≥ $\lceil m/2 \rceil$ চাইল্ড

CLRS t: $t-1 \ldots 2t-1$ কি

$h \le \log_t((n+1)/2)$

বি+: লিফ লিংকড, ডেটা লিফে

বি+ রেঞ্জ: $O(\log n + k)$

MCQ-র সেরা ১২টি এক-লাইনের কথা

1. হিপ ক্রম বাম-ডান নিয়ে কিছু বলে না
2. বিল্ড-হিপ $\lfloor n/2 \rfloor$ থেকে শুরু, নামে ১ পর্যন্ত
3. $\Sigma\, h/2^h = 2 \Rightarrow$ বিল্ড-হিপ লিনিয়ার
4. k আকারের হিপে টপ-k: $O(n \log k)$
5. মার্জযোগ্য হিপ আছে **ইউনিয়নের** জন্য
6. ক্যাসকেডিং কাট: দ্বিতীয় চাইল্ড হারালে কাট
7. ফিবোনাচ্চির সীমা **অ্যামর্টাইজড**
8. অর্ডার 5: কি 2...4, চাইল্ড 3...5
9. অর্ডার 3 = 2-3 ট্রি; অর্ডার 4 = 2-3-4 ট্রি
10. বি-ট্রি বাড়ে ও কমে **রুটে**
11. ওভারফ্লো ⇒ স্প্লিট; আন্ডারফ্লো ⇒ ধার / মার্জ
12. বি+ লিফ স্প্লিটে কি উপরে কপি হয়

প্রধান ফাঁদ: $2i+1$ বনাম $2i$ (ইনডেক্সিং দেখুন) · হিপিফাই $O(\log n)$ বনাম বিল্ড-হিপ $O(n)$ · যেকোনো হিপে "$O(1)$ সার্চ" · ফিবোনাচ্চি এক্সট্র্যাক্ট-মিন "$O(1)$" · অ্যামর্টাইজডকে ওয়ার্স্ট কেস ধরা · অর্ডার m-এ "m-টি কি" · ন্যূনতম কি = $\lceil m/2 \rceil - 1$, $\lceil m/2 \rceil$ নয় · বি-ট্রি "লিফে বাড়ে"।



সব কৌশল একসাথে: নিচে নামতে দ্বিগুণ, উপরে উঠতে অর্ধেক · বাইনোমিয়াল হিপ মানে n বাইনারিতে, ইউনিয়ন মানে যোগ · ফিবোনাচ্চি অলস: মিন বের করা ছাড়া সব $O(1)$ · এক চাইল্ড হারালে মার্ক, দুই হারালে কাট · বেশি ভরা: স্প্লিট করে উপরে ঠেলো; বেশি খালি: ধার, নইলে মার্জ।

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. ম্যাক্স-হিপে **সবচেয়ে বড়** উপাদান থাকে:
ক) সবচেয়ে বামের লিফে খ) অ্যারের শেষ ঘরে
গ) যেকোনো একটি লিফে ঘ) রুটে

২. রুট ইনডেক্স 1-এ হলে নোড i-এর চাইল্ড কোথায়?
ক) 2i ও 2i + 1 খ) 2i + 1 ও 2i + 2
গ) i + 1 ও i + 2 ঘ) i/2 ও i/2 + 1

৩. ম্যাক্স-হিপ `[50, 30, 40, 10, 20]`-এ সিফট-আপে 45 ইনসার্ট করলে অ্যারে হয়:
ক) [50,45,30,10,20,40] খ) [50,30,45,10,20,40]
গ) [45,30,50,10,20,40] ঘ) [50,30,40,10,20,45]

৪. n-টি অসাজানো কি থেকে নিচ-থেকে-উপরে বাইনারি হিপ বানাতে সময়:
ক) O(log n) খ) O(n log n)
গ) O(n) ঘ) $O(n^2)$

৫. ফিবোনাচ্চি হিপে **ডিক্রিজ-কি**-র অ্যামর্টাইজড খরচ:
ক) O(log n) খ) O(n)
গ) O(1) ঘ) O(log log n)

৬. ফিবোনাচ্চি হিপসহ ডাইকস্ট্রার অ্যালগরিদমের সময়:
ক) $O(V^2)$ খ) O(E + V log V)
গ) O((V + E) log V) ঘ) O(E log E)

৭. অর্ডার m = 5-এর বি-ট্রিতে নোডে সর্বোচ্চ কি ও নন-রুট নোডে ন্যূনতম কি:
ক) 5 ও 3 খ) 4 ও 3
গ) 5 ও 2 ঘ) 4 ও 2

৮. বি-ট্রির উচ্চতা কখন বাড়ে?
ক) রুট স্প্লিট হলে খ) লিফ স্প্লিট হলে
গ) দুটি লিফ মার্জ হলে ঘ) কি ধার করা হলে

৯. ১৩ নোডের বাইনোমিয়াল হিপে কোন ট্রিগুলো থাকে?
ক) B_3, B_1, B_0 খ) B_2, B_1, B_0
গ) B_3, B_2, B_1 ঘ) B_3, B_2, B_0

১০. ডেটাবেসের রেঞ্জ কোয়েরিতে বি-ট্রির চেয়ে বি+ ট্রি কেন পছন্দ?
ক) এটি বাইনারি ট্রি খ) সব কি লিংকড লিফে
গ) ইন্টারনাল নোড লাগে না ঘ) ডেটা রুটে রাখে

উত্তরমালা → ১-ঘ · ২-ক · ৩-খ · ৪-গ · ৫-গ · ৬-খ · ৭-ঘ · ৮-ক · ৯-ঘ · ১০-খ

লিখিত প্রশ্ন: (ক) দেখান যে BUILD-MAX-HEAP O(n) সময়ে চলে। (খ) একটি খালি **অর্ডার 5** বি-ট্রিতে 10, 20, 30, 40, 50, 60, 70, 80, 90 ইনসার্ট করুন, প্রতিটি স্প্লিট দেখান, তারপর 40 ডিলিট করুন। (গ) ফিবোনাচ্চি হিপের অ্যামর্টাইজড খরচগুলো লিখুন এবং ব্যাখ্যা করুন কেন তখন ডাইকস্ট্রা O(E + V log V)-এ চলে। **১০ নম্বর**

১ **(ক) বিল্ড-হিপ (৩)** — i = ⌊n/2⌋ … 1 হিপিফাই; h উচ্চতার নোডে খরচ O(h), এমন নোড ≤ ⌈n/2^(h+1)⌉-টি; Σ ⌈n/2^(h+1)⌉ ·O(h) = O(n Σ h/2^h) = O(2n) = O(n)। ৩

২ **(খ) স্প্লিট (৩)** — অর্ডার 5 ⇒ সর্বোচ্চ 4, ন্যূনতম 2-টি কি। 10–40 এক লিফ ভরে; 50-এ ওভারফ্লো ⇒ মধ্যমা 30 উপরে: [30] / [10 20] [40 50]। 60, 70 ⇒ [40 50 60 70]; 80-তে ওভারফ্লো ⇒ 60 উপরে: [30 60] / [10 20] [40 50] [70 80]; 90 ⇒ [70 80 90]। ৩

৩ **(খ) 40 ডিলিট (২)** — [50] আন্ডারফ্লো; বাম সিবলিং ন্যূনতমে, ডান সিবলিং [70 80 90]-এ বাড়তি ⇒ ধার: 60 নিচে, 70 উপরে ⇒ [30 70] / [10 20] [50 60] [80 90]। ২

৪ **(গ) ফিবোনাচ্চি হিপ (২)** — ইনসার্ট, ইউনিয়ন, ফাইন্ড-মিন, ডিক্রিজ-কি O(1); এক্সট্র্যাক্ট-মিন, ডিলিট অ্যামর্টাইজড O(log n)। ডাইকস্ট্রা = V-টি এক্সট্র্যাক্ট-মিন + E-টি ডিক্রিজ-কি = O(V log V + E)। ২

পরীক্ষার আগে ✓

- ✓ দুই সেট ইনডেক্স সূত্র লিখতে পারি
- ✓ হাতে অ্যারেতে সিফট-আপ / ডাউন করতে পারি
- ✓ বিল্ড-হিপ O(n) প্রমাণ করতে পারি
- ✓ তিন হিপের খরচের টেবিল পূরণ করতে পারি
- ✓ অর্ডার-m-এর ন্যূনতম/সর্বোচ্চ কি বলতে পারি
- ✓ বি-ট্রিতে স্প্লিট, ধার ও মার্জ করতে পারি